

Serial UART Communication Experiment

Serial UART Communication Experiment

- 1. UART Communication Overview
- 2. Core Concepts
 - 2.1 What is UART
 - 2.2 UART Parameter Selection
 - 2.3 Core API
- 3. Hardware Dependencies
- 4. Project Architecture and Flow
- 5. Source Code Details
 - 5.1 UART Initialization
 - 5.2 Transmit and Receive
 - 5.3 Main Loop
- 6. Operation Procedure
 - 6.1 Flashing and Running
 - 6.2 Serial Output Example
 - 6.3 Expected Behavior
 - 6.4 Verification Method
- 7. Common Issues

1. UART Communication Overview

UART (Universal Asynchronous Receiver/Transmitter) is a serial communication protocol that only needs two wires, TX (transmit) and RX (receive), to achieve bidirectional data transfer.

In this experiment, `machine.UART` is used to configure UART1 of the ESP32-S3, continuously sending a string at 115200 baud and performing loopback reception.

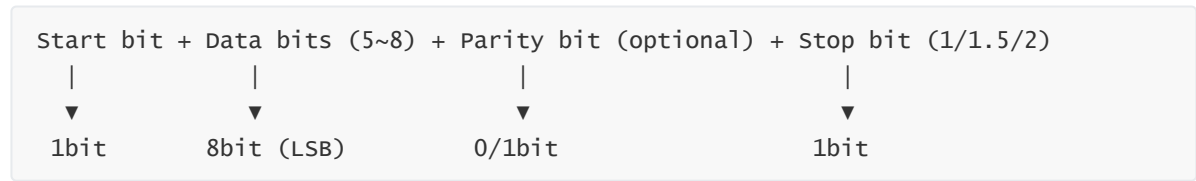
Typical application scenarios:

Application	Example
Debug output	Serial terminal, log printing
Module communication	GPS, Bluetooth module, 4G module
Device interconnection	Data exchange between MCUs
Sensor readings	Serial sensors (PM2.5, temperature/humidity, etc.)

2. Core Concepts

2.1 What is UART

UART converts parallel data into a serial bit stream, transmitted bit by bit according to the frame format:



The common format **8N1** = 8 data bits, no parity, 1 stop bit (MicroPython default).

2.2 UART Parameter Selection

Parameter	Value in this experiment	Description
Port	UART1	Avoids UART0 (occupied by the REPL system)
Baud rate	115200	Standard rate, compatible with most devices
TX pin	GPIO10	Transmit data line
RX pin	GPIO11	Receive data line
Data bits	8 (default)	8 data bits
Parity	None (default)	No parity

ESP32-S3 has 3 UARTs: UART0 (occupied by REPL), UART1 and UART2 (available to users).

2.3 Core API

```
from machine import UART, Pin

uart = UART(1, baudrate=115200, tx=Pin(10), rx=Pin(11, Pin.IN, Pin.PULL_UP)) #
Create UART, RX pull-up
uart.write(b'hello\r\n') # Send byte string
buf = uart.read(255) # Read up to 255 bytes
data = buf.decode('utf-8').strip() # Decode into a string
```

3. Hardware Dependencies

GPIO	Peripheral	Mode	Description
10	UART1 TX	UART	Transmit pin
11	UART1 RX	UART	Receive pin

Loopback test: Short GPIO10 (TX) and GPIO11 (RX) with a Dupont wire to transmit and receive on its own.

Wiring diagram:

4. Project Architecture and Flow

```
Script execution (single-file .py)
|
├─ UART(1, baudrate=115200, tx=10, rx=11)
|
└─ while True:
    │ print("Sent: ...")           // Print sent data
    │ uart.write(...)             // Send
    │ sleep_ms(50)                 // wait for data to fully
arrive
    │ uart.read(255) → decode('utf-8') → print("Received:") // Receive and
print
    │ sleep_ms(2000)               // 2s interval
```

5. Source Code Details

Complete source code: [5.Source Code](#) → [MicroPython](#) → [3.Serial Communication and Storage System](#) → [1. UART](#) → [1. UART.py](#)

5.1 UART Initialization

```
from machine import UART, Pin # UART and Pin classes
import time                  # delay

# Initialize UART1 (avoid REPL UART0)
# RX pull-up to avoid garbage when floating
uart = UART(1, baudrate=115200, tx=Pin(10), rx=Pin(11, Pin.IN, Pin.PULL_UP))
```

5.2 Transmit and Receive

```
# TX function
def uart_tx(data):
    uart.write(data)

# RX function
def uart_rx():
    buf = uart.read(255) # Read serial buffer, max 255 bytes
    if buf:
        try:
            print("Received:", buf.decode('utf-8').strip()) # Normal text
decode
        except UnicodeError:
            print("Received raw:", buf) # Print raw bytes on failure
```

5.3 Main Loop

```
# Main logic
print("UART Started, TX=GPIO10, RX=GPIO11")
while True:
    send_str = "GOOD GOOD STUDY, DAY DAY UP!!!!!!\r\n"
    print("Sent:", send_str.strip())      # Print what was sent
    uart_tx(send_str)
    time.sleep_ms(50)                    # Brief delay for RX to complete
    uart_rx()
    time.sleep_ms(2000)                  # 2s interval
```

6. Operation Procedure

6.1 Flashing and Running

Thonny IDE operation steps: See `MicroPython` → 1. Before Development → 3. Thonny IDE.

1. **Loopback test:** Short GPIO10 (TX) and GPIO11 (RX) with a Dupont wire
2. Open `1. UART.py` with Thonny IDE and click Run

6.2 Serial Output Example

```
UART Started, TX=GPIO10, RX=GPIO11
Sent: GOOD GOOD STUDY, DAY DAY UP!!!!!!
Received: GOOD GOOD STUDY, DAY DAY UP!!!!!!
Sent: GOOD GOOD STUDY, DAY DAY UP!!!!!!
Received: GOOD GOOD STUDY, DAY DAY UP!!!!!!
...
```

```
UART Started, TX=GPIO10, RX=GPIO11
Sent: GOOD GOOD STUDY, DAY DAY UP!!!!!!
Received: GOOD GOOD STUDY, DAY DAY UP!!!!!!
Sent: GOOD GOOD STUDY, DAY DAY UP!!!!!!
Received: GOOD GOOD STUDY, DAY DAY UP!!!!!!
Sent: GOOD GOOD STUDY, DAY DAY UP!!!!!!
Received: GOOD GOOD STUDY, DAY DAY UP!!!!!!
```

6.3 Expected Behavior

Prints the received string every 2 seconds. If TX/RX is shorted correctly, the `Received` content is identical to the sent content.

6.4 Verification Method

Action	Expected result
Short GPIO10 - GPIO11	<code>Received:</code> shows the complete sent content
Remove the shorting wire	No <code>Received:</code> output (no data received)
Change <code>baudrate=9600</code>	Garbled data (TX/RX baud rates do not match)
Change <code>uart.write("Hello")</code>	Received content becomes "Hello"

7. Common Issues

Symptom	Cause	Solution
No Received output	TX/RX not shorted	Connect GPIO10 and GPIO11 with a Dupont wire
Received shows garbage or abnormal characters	Baud rate mismatch	Confirm <code>baudrate=115200</code> is the same on both ends
<code>UART(0, ...)</code> fails	UART0 is occupied by the system REPL	Use <code>UART(1)</code> or <code>UART(2)</code> instead
Received is empty after sending	Insufficient receive delay	Increase the <code>sleep_ms()</code> delay